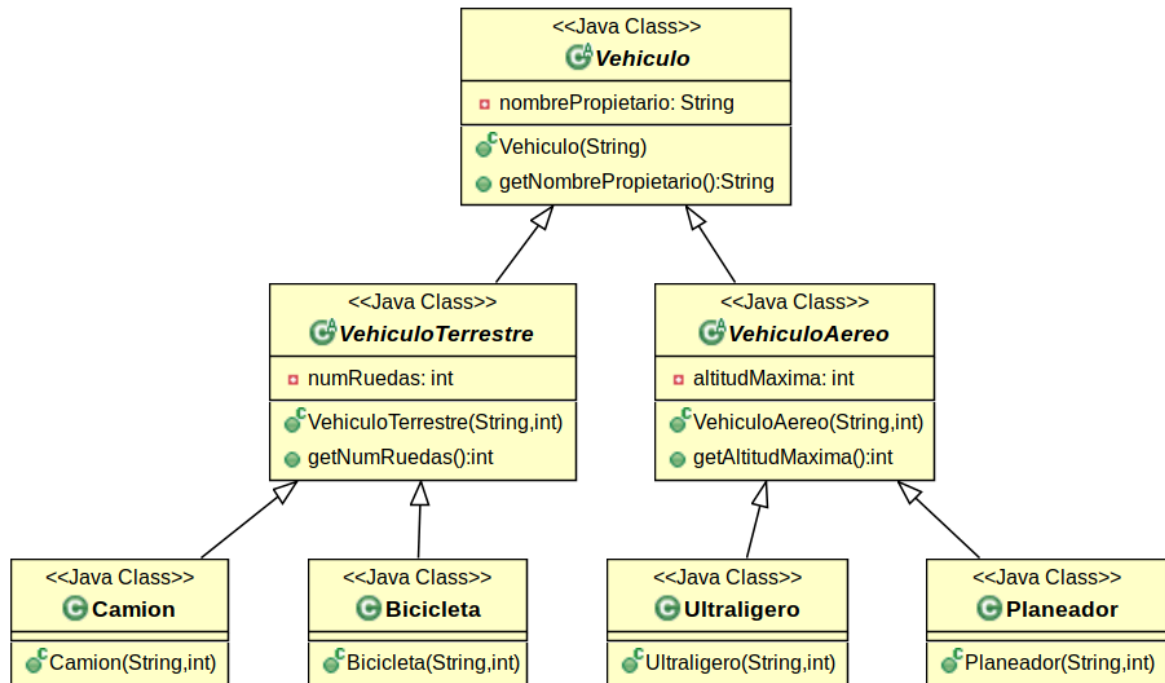


UT 07 – Ejercicios

Ejercicio 22 – Clases, interfaces, herencia, excepciones.

A

Implementar la jerarquía de clases que aparecen en el siguiente diagrama.



La leyenda de los símbolos en el diagrama es la siguiente:

- Un cuadrado rojo (■) representa un atributo privado (símbolo – en UML).
- Un círculo verde (●) representa un método público (símbolo + en UML). Una C sobre el indicador sólo aparece en los constructores.
- Un círculo verde con una C dentro indica clase. Si tiene una A sobre el indicador, es que es clase abstracta.
- Las líneas continuas acabadas en flecha indican herencia.

Teniendo en cuenta que:

- El nombre del propietario no puede ser null ni cadena vacía.
- El número de ruedas de un vehículo terrestre no puede ser:
 - o Para los camiones: menor que cuatro ni mayor que 16.
 - o Para las bicicletas, distinto de 2.
- La altitud máxima de cualquier vehículo aéreo no puede ser menor que 100 ni mayor que 10000.
- Si se intenta crear un vehículo con un atributo no válido, se tiene que lanzar `InvalidArgumentException` con un mensaje descriptivo.

B

Una vez implementado, añadir / cambiar lo necesario para que:

- Los objetos de la clase “Camion” tengan una propiedad para almacenar su carga máxima. Esta carga máxima no tendrá decimales y será siempre mayor que 0 e inferior a 40000. Esta propiedad, desde fuera de la clase, se podrá consultar, pero no modificar.
- Los objetos de la clase “Ultraligero” tengan una propiedad para almacenar la envergadura. Esta envergadura tendrá decimales y será siempre mayor que 5 e inferior a 20 metros. Esta propiedad, desde fuera de la clase, se podrá consultar, pero no modificar.
- Las clases Camión y Ultraligero modificarán sus constructores para poder recibir respectivamente, el valor de la carga máxima y de la envergadura.
- Si se llama al constructor con valores no válidos, se lanzará una excepción del tipo “InvalidArgumentException” con un mensaje indicando el problema.

C

Crear las siguientes interfaces:

- Matriculable. Con un solo método: String getMatricula().
- Motorizado. Con un solo método: String getPotencia().

Implementar las interfaces en las clases, añadiendo los atributos necesarios y modificando los constructores, para que se implementen de la siguiente forma:

- Clase Planeador: sólo la interfaz Matriculable.
- Clases Ultraligero y Camion: las dos interfaces.